

ERP Developer Guide

Project: Wholesale & Distribution ERP

Document status: Current development and operations guide

Last updated: August 12, 2026

Primary architecture: Multi-tenant modular monolith

Primary audience: Developers, technical leads, DevOps engineers, reviewers, and future ChatGPT development sessions

Guide Map

Use this map to jump conceptually to the right part of the guide:

- **Start here:** Purpose, technology stack, and the August 12 working-state addendum.
 - **Security boundaries:** Multi-tenancy, branch access, Spatie Permission, Platform/Super Admin separation, SaaS enforcement.
 - **Coding rules:** PHP standards, no-enum rule, Vue/TypeScript standards, Form Requests, common responses, decimal rules.
 - **Transactional design:** database transactions, locking, document numbering, accounting invariants, inventory invariants.
 - **Business modules:** organisation, master data, purchasing, sales, inventory, AP/AR, treasury, reporting, operations.
 - **Frontend and routing:** route organization, Inertia page conventions, SSR/hydration notes, numeric form normalization.
 - **Operations:** queues, scheduler, exports, backups, health checks, production readiness, preflight and acceptance.
 - **Deployment:** release-candidate freeze, maintenance-mode cutover, post-deployment verification.
 - **Audit and maintenance:** security, immutability, migration rules, route/module audits, current audit status.
 - **Handoff:** development workflow, command reference, engineering principles, developer handoff checklist, source-of-truth priority.
-

1. Purpose

This document is the working developer guide for the current ERP codebase.

Use it to:

- understand the project architecture;
- continue development without rebuilding completed modules;
- preserve tenant and branch isolation;
- follow the project's PHP, Laravel, Vue, and TypeScript conventions;
- implement transactional accounting and inventory workflows safely;
- run local development and production verification;
- understand production acceptance and release-candidate controls;
- onboard another developer to the project.

When this guide conflicts with the actual current project code, **the latest project code is authoritative**. Inspect the current project before modifying existing functionality.

2. Technology Stack

The current ERP uses:

- PHP 8.4
- Laravel 13
- MySQL
- Inertia.js 3
- Vue 3 Composition API
- TypeScript in strict mode
- Tailwind CSS 4
- TailAdmin-based ERP interface
- Spatie Laravel Permission with tenant teams
- Database-backed / asynchronous queues
- Laravel scheduler
- Vite
- Brick Math for precise decimal arithmetic where required

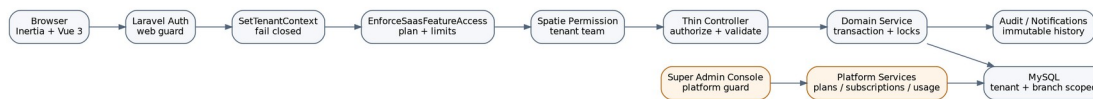
Application interfaces use **Inertia + Vue**.

Blade is not used for normal application pages. It is limited to infrastructure-level Laravel needs such as the Inertia root shell.

Current Working-State Addendum - August 12, 2026

This section records architecture and compatibility rules added or confirmed during the latest manual acceptance cycle. It should be read before modifying purchasing, product configuration, Platform/SaaS, or shared frontend infrastructure.

A. Request and service architecture at a glance



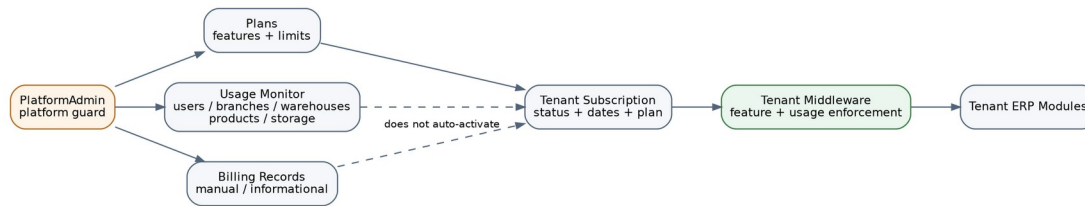
The normal tenant ERP request path is intentionally different from the Platform/Super Admin path. Tenant requests establish tenant context, enforce SaaS feature access, apply tenant-aware permissions, and then enter thin controllers and transactional services. Platform requests use the separate platform guard and explicit platform services.

Do not merge these authentication domains. A Platform Administrator is not a tenant User, and tenant users.tenant_id must not be made nullable to support Super Admin access.

B. Platform / Super Admin architecture

The project now includes a separate Super Admin Console for SaaS operations. Its current responsibilities include:

- Platform dashboard;
- tenant management and tenant activation/suspension;
- SaaS plan CRUD-style management;
- tenant subscription assignment and date/status management;
- quick manual renewal/extension actions;
- tenant usage monitoring;
- subscription change history;
- manual/informational billing records.



Authentication boundary

Platform access uses a dedicated PlatformAdmin model / platform guard. Keep Platform routes, middleware, layouts, and controllers separate from tenant ERP authentication.

Manual SaaS model

The current SaaS design is deliberately manual.

- A Super Admin may assign a plan and subscription state directly.
- An active subscription does **not** require a paid invoice.
- Billing records do **not** automatically activate or deactivate a tenant.
- No payment gateway is part of the current acceptance scope.
- Subscription dates/status and package allocation are controlled by Super Admin operations.

Do not re-introduce payment-dependent access unless the product requirements explicitly change.

Current feature catalog

Boolean module entitlements currently include:

```

master_data.module
purchasing.module
sales.module
inventory.module
inventory.advanced
accounts_payable.module
accounts_receivable.module
financial_accounting.module
treasury.module
management_reporting.module
audit_exports.module
  
```

Usage limits currently include:

```

users.limit
branches.limit
warehouses.limit
products.limit
storage_mb.limit
  
```

A blank enabled limit means unlimited. Feature and usage enforcement must remain tenant-scoped and must fail closed when required subscription context is invalid.

C. Product availability is location-aware

Creating a product master record alone does not make the item available in every transactional selector. Product eligibility is intentionally layered.

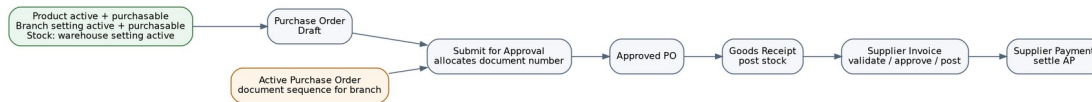
For purchasing, a product normally needs:

1. the Product itself to be active;
2. the Product itself to be purchasable;
3. an active Product Branch Setting for the PO branch;

4. that branch setting to be purchasable;
5. for stock products, an active Product Warehouse Setting for the receiving warehouse.

This rule explains why a newly created product can exist in **Products** but not appear under **Purchase Order -> Order Lines**. Do not bypass this filtering by returning every tenant product. Fix missing configuration or make the UI explain the prerequisite.

D. Purchase Order lifecycle and prerequisites



A Purchase Order may be saved in draft before its final document number is allocated. On **Submit for Approval**, the service requires an active document-number sequence for the selected branch and the purchase_order document type.

If the sequence is missing, the correct behavior is a validation failure such as:

No active document-number sequence is configured for this document type and branch.

Do not bypass this check with ad-hoc numbers. Configure or provision the proper branch sequence.

A future branch-provisioning improvement should initialize required document sequences for newly created branches, or the UI should explicitly guide the administrator to configure them.

E. Laravel 13 compatibility notes confirmed during acceptance

orWhereKey()

Laravel 13 Eloquent Builder does not provide `orWhereKey()`. Code such as:

```
$query->orWhereKey($id);
```

must be replaced with an explicit qualified-key condition, for example:

```
$query->orWhere(
    $query->getModel()->getQualifiedKeyName(),
    $id,
);
```

This matters particularly in edit-form option queries where the current branch, warehouse, supplier, customer, or related record must remain selectable even if it no longer matches the default active filter.

Relation callback typing

When constraining a relationship passed to `load()`, type the callback to the actual relation when strict typing requires it. Example:

```
use Illuminate\Database\Eloquent\Relations\HasMany;

$fiscalYear->load([
    'periods' => static function (HasMany $relation): void {
        $relation->orderBy('period_number');
    },
]);
```

Typing such callbacks as `Eloquent\Builder` can fail because Laravel passes a relation object.

F. Brick Math 0.18 rounding-mode names

The project uses Brick Math for precise decimal arithmetic. With the installed modern Brick Math API, use PascalCase enum cases:

```
RoundingMode::Unnecessary
RoundingMode::HalfUp
RoundingMode::Down
```

Do not use legacy uppercase names such as:

```
RoundingMode::UNNECESSARY
RoundingMode::HALF_UP
RoundingMode::DOWN
```

Unnecessary is useful for validation because it rejects values that would require rounding instead of silently changing user input.

When upgrading dependencies, search the full application for deprecated API names instead of fixing only the first stack trace.

G. Vue numeric-input normalization

HTML/Vue numeric inputs can produce runtime numbers even when initial form data was supplied as strings. Therefore this is unsafe:

```
form.credit_limit = form.credit_limit.trim();
line.discount_amount = line.discount_amount.trim();
```

For editable numeric form fields, prefer a form type such as:

```
type NumericFormValue = string | number;
```

and normalize safely before submission:

```
const normalized = String(value).trim();
```

This pattern has already affected Customer, Product, and Purchase Order forms. When a similar `.trim` is not a function error appears, audit the **whole form** for numeric fields rather than patching only the field named in the first exception.

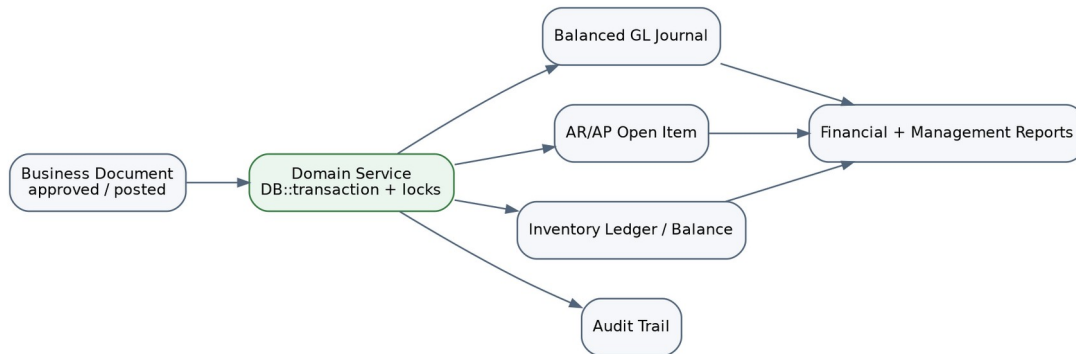
H. SSR / hydration rules

Shared UI components mounted in both tenant and Platform layouts must be SSR-safe. Browser-only behavior, teleports, and client-state initialization should not produce different initial server/client markup.

For example, toast/teleport UI should not become visible during SSR based on state that only exists on the client. Initialize display behavior after `onMounted()` when appropriate.

When Vite SSR reports a page cannot be found, verify the exact page path and case under `resources/js/Pages/...`, clear Vite caches, and restart the dev processes before changing unrelated business logic.

I. Accounting / inventory posting principle



Posting services should remain the single transactional owner of their financial and stock side effects. A business document that posts should update all required journal/open-item/stock/audit state atomically. Never let controllers partially reproduce posting logic.

J. Deployment and release flow



Production acceptance is not only a route smoke test. It must verify tenant isolation, branch integrity, balanced accounting, AR/AP reconciliation, inventory balance, reservations, critical routes, worker/scheduler health, and release-candidate integrity.

3. Architectural Principles

3.1 Modular monolith

The ERP is implemented as a Laravel modular monolith.

Business areas are separated through:

- models;
- controllers;
- Form Requests;
- services;
- policies;
- support registries;
- route files;
- Vue pages;
- TypeScript types;
- scheduled commands;
- queue jobs;
- operational services.

Avoid unnecessary microservices unless a future requirement clearly justifies them.

3.2 Thin controllers

Controllers should:

- authorize;
- receive validated input;
- call domain/application services;
- return Inertia or the common response format.

Controllers should not contain complex:

- accounting logic;
- inventory calculations;
- transaction orchestration;
- concurrency rules;
- status-transition rules.

3.3 Service-layer workflows

Business workflows belong in services.

Use services for:

- document posting;
- reversals;
- allocations;
- settlement;
- reservations;
- stock movements;
- accounting journal generation;
- approval transitions;
- reconciliation;
- reporting calculations;
- operational integrity checks.

3.4 Fail closed

For financial, inventory, authorization, and tenancy workflows:

When required context or required accounting/inventory infrastructure is missing, fail closed.

Do not create incomplete financial records merely to allow a workflow to continue.

4. Mandatory PHP Standards

Every PHP file must begin with:

```
<?php
```

```
declare(strict_types=1);
```

Follow:

- PSR-12;
- PSR-4;

- Laravel conventions;
- typed parameters;
- typed properties;
- explicit return types;
- constructor dependency injection;
- constructor property promotion where useful;
- final classes when inheritance is unnecessary;
- useful PHPDoc for arrays and Eloquent relations.

Example:

```
<?php

declare(strict_types=1);

namespace App\Services\Example;

final class ExampleService
{
    public function execute(int $tenantId): void
    {
        // ...
    }
}
```

5. No-Enum Rule

Do **not** use:

- PHP enums;
- database ENUM columns;
- TypeScript enums.

Use strings instead.

For database status/type fields:

```
$table->string('status', 30)
    ->default('draft')
    ->comment('draft, approved, posted, reversed, cancelled')
    ->index();
```

Validate allowed values using:

- Form Requests;
- service-layer transition rules;
- string status registries/constants where appropriate;
- TypeScript string union types.

Example:

```
type InvoiceStatus =
    | 'draft'
    | 'approved'
    | 'posted'
    | 'reversed'
    | 'cancelled';
```


6. Vue and TypeScript Standards

Use Vue 3 Composition API:

```
<script setup lang="ts">
```

Requirements:

- strict TypeScript;
- never use any;
- type all props;
- type forms;
- type filters;
- type pagination;
- type API/Inertia payloads;
- type refs and computed values;
- type action functions;
- use string unions instead of TypeScript enums.

Reuse existing project components and patterns, including:

- ErpLayout;
- current TailAdmin styling;
- dark-mode classes;
- Inertia Link;
- Inertia router;
- Inertia useForm;
- the existing authorization composable;
- existing reusable table/form/input components where available.

For large forms, use the existing `Partials` convention.

Do not introduce a second UI framework.

7. Multi-Tenancy

7.1 Tenant definition

A tenant represents a separate company/business using the ERP.

Example:

```
Tenant 1 = Company A
Tenant 2 = Company B
```

If only one company currently uses the ERP, Tenant 1 is enough.

Tenant 2 is needed when:

- a second company uses the same ERP installation;
- multi-company behavior is required;
- tenant-isolation testing is required.

7.2 Tenant rules

Every tenant-owned table must contain:

```
tenant_id
```

Every tenant-owned query must be scoped to the active tenant.

Never trust a frontend-submitted `tenant_id` when it can be obtained from `TenantContext`.

Use the existing:

```
App\Support\Tenancy\TenantContext
```

and the project's existing tenant model conventions.

Where appropriate, tenant-owned models use the existing tenant trait/convention such as `BelongsToTenant`.

7.3 Fail-closed tenant scoping

Tenant-scoped models and services must fail closed when tenant context is unavailable.

Never make a missing tenant context mean:

```
show all tenants
```

It should instead mean:

```
deny / fail
```

7.4 Tenant-safe unique constraints

Tenant-specific uniqueness should normally include `tenant_id`.

Example:

```
$table->unique([
    'tenant_id',
    'code',
]);
```

7.5 Tenant-safe background work

Queue jobs, scheduled commands, exports, reports, files, notifications, and cache entries must preserve tenant identity.

Background jobs should:

1. receive or resolve the tenant ID;
 2. establish `TenantContext`;
 3. establish the Spatie permission team where required;
 4. process only that tenant;
 5. clear tenant/team context in `finally`.
-

8. Branch-Level Access

Tenant isolation and branch access are separate controls.

A user may belong to Tenant 1 but only have access to selected branches inside Tenant 1.

Use the existing branch-access service and its actual project API.

Previously established methods include patterns such as:

```
hasCompanyWideAccess()
canAccessBranch()
authorizeBranch()
findAccessibleBranch()
scopeQuery()
scopeBranchQuery()
accessibleBranches()
```

Before calling a method, inspect the current BranchAccessService implementation and use its real signature.

Never implement branch filtering only in Vue.

Backend authorization must enforce it.

9. Spatie Permission

Spatie Laravel Permission is configured with teams enabled.

The team foreign key is:

```
tenant_id
```

When a tenant is active, the correct permission team must also be active.

Typical command/service flow:

```
$tenantContext->set($tenant);

$permissionRegistrar->setPermissionsTeamId(
    (int) $tenant->getKey(),
);

try {
    // tenant-scoped operation
} finally {
    $permissionRegistrar->setPermissionsTeamId(null);
    $tenantContext->clear();
}
```

Frontend permission checks control visibility only.

Backend authorization remains mandatory.

10. Common Response Service

Use the project's existing common response service for write responses.

Current project location:

```
app/Support/Responses/CommonResponseService.php
```

Expected successful JSON shape:

```
{
    "success": true,
    "message": "Operation completed successfully.",
    "data": {},
}
```

```
"meta": {}
}
```

Expected error shape:

```
{
  "success": false,
  "message": "The request could not be completed.",
  "errors": {},
  "code": "REQUEST_FAILED"
}
```

For Inertia requests, preserve the existing flash/toast behavior.

Do not introduce inconsistent response formats without a strong framework-specific reason.

11. Form Requests and Validation

Use Form Requests for request validation when that matches the existing module pattern.

Validation is not enough for business invariants.

Use:

- Form Requests for input shape and allowed values;
- policies/permissions for authorization;
- services for state transitions and business rules.

Never trust:

- frontend totals;
 - frontend-calculated taxes;
 - frontend-calculated discounts;
 - frontend stock availability;
 - frontend tenant identity;
 - frontend branch identity without server authorization.
-

12. Money and Quantity Rules

Never use floating-point database fields for financial values.

Use decimal columns with appropriate precision and scale.

Backend services must recalculate totals.

Use precise decimal arithmetic where financial correctness requires it.

Typical values requiring consistent precision:

- quantities;
- unit prices;
- discounts;
- taxes;
- exchange rates;
- document totals;
- ledger amounts;

- inventory valuation.

Posted financial or inventory records must not be silently edited or deleted.

Use:

- explicit reversal;
 - compensating transactions;
 - audited correction workflows.
-

13. Database Transactions and Concurrency

Use database transactions for workflows that affect multiple related records.

Typical transactional workflows include:

- posting supplier invoices;
- posting sales invoices;
- customer receipts;
- supplier payments;
- customer refunds;
- credit applications;
- AR/AP adjustments;
- stock allocation;
- stock movement;
- purchase returns;
- sales returns;
- treasury transfers;
- bank reconciliation.

When competing updates are possible, use row locking.

Example:

```
DB::transaction(function (): void {
    $record = SomeModel::query()
        ->lockForUpdate()
        ->findOrFail($id);

    // validate current state
    // update atomically
});
```

Do not rely only on frontend button disabling for concurrency safety.

14. Document Numbering

The ERP uses centralized document numbering.

Use the existing:

```
DocumentNumberService
DocumentTypeRegistry
document_sequences
```

Do not hard-code transactional document numbers.

A tenant should have an active sequence for every registered transactional document type required by its workflows.

Document numbering coverage is part of production acceptance.

15. Accounting Architecture

The ERP contains General Ledger, Accounts Receivable, Accounts Payable, treasury/banking, reporting, and settlement workflows.

Core accounting safety rules:

- postings must be atomic;
- journals must balance;
- journal lines must belong to the same tenant and correct branch;
- header totals must reconcile to line totals;
- posted records are immutable;
- reversals must be explicit;
- open-item arithmetic must reconcile;
- settlement allocations must not cross tenant/branch/party ownership.

15.1 General Ledger invariant

For every posted journal:

```
header base debit = header base credit
```

and:

```
sum(line base debit) = sum(line base credit)
```

and:

```
sum(line base debit) = header base debit
sum(line base credit) = header base credit
```

A posted journal should contain the minimum required line structure for a valid double-entry posting.

15.2 Correct journal-line fields

The current journal-line schema uses:

```
base_debit_amount
base_credit_amount
```

Do not use obsolete/nonexistent names such as:

```
base_debit
base_credit
```

15.3 AR/AP open-item invariant

For customer and supplier open items:

```
allocated_amount + outstanding_amount = original_amount
```

and in base currency:

```
base_allocated_amount
+ base_outstanding_amount
= base_original_amount
```

Open-item amounts must not become negative through normal workflows.

15.4 Settlement ownership

Customer allocations must stay within:

```
same tenant
same branch
same customer
```

Supplier allocations must stay within:

```
same tenant
same branch
same supplier
```

16. Inventory Architecture

Inventory balances and reservation workflows must remain consistent.

Current inventory balance integrity focuses on fields such as:

```
quantity_on_hand
inventory_value
average_unit_cost
version
```

Do not assume a generic quantity_reserved column exists on inventory_balances.

Reservation logic may live in workflow-specific domains.

16.1 Purchase Return reservation invariant

Goods Receipt return state uses:

```
accepted_quantity
returned_quantity
return_reserved_quantity
```

Required invariant:

```
returned_quantity
+ return_reserved_quantity
<= accepted_quantity
```

All three values must remain non-negative.

16.2 Inventory balance invariant

For normal production acceptance:

```
quantity_on_hand >= 0
inventory_value >= 0
```

and a zero on-hand balance should not retain an unexplained non-zero inventory value.

Any correction should be made through an audited reconciliation process rather than direct manual editing of posted stock data.

17. Major Business Areas

The current ERP codebase includes or has foundations for the following areas.

Organisation and Administration

- tenants;
- company settings;
- branches;
- warehouses;
- users;
- roles;
- permissions;
- currencies;
- exchange rates;
- taxes;
- accounting/fiscal periods;
- document numbering.

Master Data

- products;
- product categories;
- brands;
- units;
- suppliers;
- customers;
- related addresses/configuration.

Purchasing

Representative workflows include:

```
Purchase Order
-> Goods Receipt
-> Supplier Invoice
-> Supplier Payment
```

Related functionality includes purchase returns, supplier debit/credit settlement logic, AP open items, and accounting integration.

Sales

Representative workflows include:

```
Sales Order
-> Allocation
-> Dispatch
-> Sales Invoice
-> Customer Receipt
```

Related functionality includes:

- sales returns;
- customer credit notes;
- customer refunds;

- credit applications;
- AR adjustments;
- customer open-item settlement.

Inventory

Includes:

- warehouse stock;
- balances;
- stock ledger;
- reservations;
- returns;
- inventory valuation;
- operational integrity controls.

Accounting and Treasury

Includes:

- General Ledger;
- Accounts Receivable;
- Accounts Payable;
- customer/supplier subledgers;
- treasury transfers;
- banking;
- bank statements;
- bank reconciliation;
- financial reporting.

Reporting and Operations

Includes:

- management reports;
 - financial statements;
 - queued exports;
 - operational health;
 - production readiness;
 - deployment preflight;
 - production acceptance;
 - backups;
 - release-candidate verification.
-

18. Routing

The ERP uses Laravel route files and route service providers to keep modules organized.

When adding a route:

1. use existing module route files;
2. use the established URL prefix;
3. use the established route-name prefix;

4. attach correct authentication/tenant middleware;
5. attach permission middleware where required;
6. verify route-model binding remains tenant safe.

After route changes:

```
php artisan optimize:clear
php artisan route:list
```

Production acceptance checks critical named routes and duplicate route names.

19. Frontend Page Conventions

Normal ERP pages use:

```
resources/js/Pages/...
```

Use module-specific page folders.

Typical pattern:

```
resources/js/Pages/PurchaseOrders/
  Index.vue
  Create.vue
  Edit.vue
  Show.vue
  Partials/
```

Reuse project layouts and components instead of copying UI markup between modules.

List screens should generally use server-side:

- search;
 - filtering;
 - sorting;
 - pagination.
-

20. Queue Architecture

The ERP uses queued/background work for operations such as exports and notifications.

Production should use a durable asynchronous queue rather than relying on synchronous execution.

Example worker:

```
php artisan queue:work \
  --queue=notifications,exports,default \
  --tries=3 \
  --timeout=1200
```

Run the worker under:

- Supervisor;
- systemd;
- another production-grade process manager.

After deployments where code changed:

```
php artisan queue:restart
```

The ERP contains operational checks for queue-worker health/heartbeat.

21. Scheduler

Laravel's scheduler must run continuously in production.

Typical cron:

```
* * * * * cd /path/to/erp && php artisan schedule:run >> /dev/null 2>&1
```

Development alternative:

```
php artisan schedule:work
```

Inspect configured jobs using:

```
php artisan schedule:list
```

Avoid duplicate scheduler registrations.

22. Exports

The ERP supports queued exports.

XLSX generation requires PHP Zip / ZipArchive.

Check:

```
php -r "var_dump(class_exists('ZipArchive'));"
```

or:

```
php -m | grep -i zip
```

CSV export does not necessarily require the same XLSX infrastructure, but supported production features should be verified before cutover.

23. Local Development Setup

23.1 Install dependencies

```
composer install
npm ci
```

23.2 Environment

```
cp .env.example .env
php artisan key:generate
```

Configure:

- application URL;
- MySQL connection;

- queue connection;
- cache/session;
- mail/notification settings as required.

23.3 Database

```
php artisan migrate
php artisan db:seed --class=DefaultChartOfAccountsSeeder
php artisan db:seed --class=PermissionSeeder
php artisan permission:cache-reset
```

23.4 Clear caches

```
php artisan optimize:clear
```

23.5 Frontend verification

```
npm run type-check
npm run build
```

23.6 Storage

When required:

```
php artisan storage:link
```

23.7 Development processes

Preferred:

```
composer run dev
```

Or run individually:

```
php artisan serve
php artisan queue:work --queue=notifications,exports,default --tries=3 --timeout=1200
php artisan schedule:work
npm run dev
```

24. Development Workflow

For each new ERP feature:

1. inspect the current code first;
2. confirm the feature is not already implemented;
3. inspect related migrations/models/services/routes/pages;
4. follow existing patterns;
5. implement one complete logical step;
6. preserve tenant and branch isolation;
7. preserve existing accounting/inventory behavior;
8. run syntax/type checks;
9. clear caches when routes/configuration changed;
10. manually validate the feature.

Do not restart completed modules from generic Laravel examples.

The current project always takes priority over older notes or historical prompts.

25. Preferred File Delivery / Review Style

When reviewing or developing this project:

- group files by category when practical;
- share complete files when several parts changed;
- use exact replacement blocks for very small changes;
- keep Controllers together;
- keep Models together;
- keep Services together;
- keep Requests together;
- keep Vue pages together;
- keep routes/configuration together.

Do not mix unrelated file types randomly.

26. Testing and Verification Policy

Automated tests are not added unless specifically requested.

Even when automated tests are not requested, developers should run appropriate static/runtime verification such as:

```
php -l path/to/File.php
npm run type-check
php artisan route:list
php artisan schedule:list
php artisan optimize:clear
```

For major workflows, manually verify:

- authorization;
 - tenant isolation;
 - branch restrictions;
 - transaction rollback behavior;
 - duplicate/retry safety;
 - accounting impact;
 - inventory impact;
 - posting/reversal behavior.
-

27. Production Server Requirements

At minimum verify:

```
php -m
```

The production environment requires Laravel's normal PHP extensions plus ERP-specific capabilities.

XLSX exports require:

```
ZipArchive
```

Database backup/restore tools require:

```
which mysqldump
which mysql
```

Production infrastructure should include:

- PHP;
 - MySQL;
 - queue worker;
 - scheduler;
 - web server;
 - process manager;
 - backup storage;
 - application monitoring/logging.
-

28. Backup Operations

Create and verify a backup:

```
php artisan erp:backup:create --verify
```

Verify an existing backup:

```
php artisan erp:backup:verify BACKUP_ID
```

Check isolated restore-drill readiness:

```
php artisan erp:backup:restore-check BACKUP_ID
```

Never perform a restore drill against the live production database.

Use a disposable database/environment.

29. Operations Health

Check runtime operations:

```
php artisan erp:health
```

Important operational concerns include:

- queue worker health;
- scheduler health;
- database availability;
- application configuration;
- storage;
- background processing;
- production dependencies.

Failed queue-job maintenance is CLI-controlled.

Typical operations include:

```
php artisan erp:queue:retry <failed-job-uuid>
php artisan erp:queue:forget <failed-job-uuid> --force
```

Investigate the underlying failure before retrying.

30. Production Readiness

Run:

```
php artisan erp:production-readiness --tenant=1
```

For all active tenants where supported:

```
php artisan erp:production-readiness
```

Readiness checks validate production prerequisites and important ERP integrity requirements.

A blocking failure means production is not ready.

Warning-only findings may be deferred according to the current project decision, but they should still be reviewed before a mature production rollout.

31. Deployment Preflight

Run:

```
php artisan erp:deploy:preflight --tenant=1
```

or for the intended active tenant set:

```
php artisan erp:deploy:preflight
```

Preflight combines the major operational/readiness/security layers.

It should fail closed when:

- a requested active tenant does not exist;
 - no active tenant is available to validate;
 - a blocking readiness/health/security condition exists.
-

32. Production Acceptance

Production Acceptance is the final persisted ERP acceptance gate.

For Tenant 1:

```
php artisan erp:acceptance --tenant=1
```

Machine-readable output:

```
php artisan erp:acceptance --tenant=1 --json
```

For multi-tenant deployment, repeat for every tenant included in cutover.

A tenant is accepted only when the report shows:

```
PASSED  
0 blocking failures
```

Do not continue to production cutover while a blocking acceptance failure remains.

33. Production Acceptance Integrity Domains

The acceptance layer validates critical areas including:

- production readiness;
 - operational health;
 - security hardening;
 - required project files;
 - required named routes;
 - duplicate route names;
 - permission coverage;
 - document-number sequence coverage;
 - tenant/branch ownership;
 - General Ledger integrity;
 - AR open-item arithmetic;
 - AP open-item arithmetic;
 - customer settlement ownership;
 - supplier settlement ownership;
 - inventory balance integrity;
 - Purchase Return reservation integrity;
 - acceptance persistence;
 - dependency lock files;
 - production Vite build manifest;
 - applied migration state;
 - release fingerprint capture.
-

34. Release Candidate Freeze

A release candidate freezes the exact accepted ERP build.

After acceptance passes:

```
php artisan erp:release:freeze 1.0.0-rc.1 --tenant=1
```

Immediately verify:

```
php artisan erp:release:verify --tenant=1
```

The frozen fingerprint covers important release material such as:

- application source;
- routes;
- migrations;
- applied migration state;
- permissions;
- backend dependency lock;
- frontend dependency lock;
- production Vite manifest.

Do not modify the accepted build and then deploy the old release candidate.

If code/configuration included in the fingerprint changes:

1. finish the intended changes;
 2. rebuild assets if required;
 3. rerun production acceptance;
 4. freeze a new RC version;
 5. verify the new RC.
-

35. Release Candidate Tenant Isolation

Release-candidate records are tenant scoped.

The following ownership must remain consistent:

```
Active Tenant
=
Production Acceptance Tenant
=
Release Candidate Tenant
```

Tenant A must never:

- freeze against Tenant B's acceptance;
- verify Tenant B's release candidate;
- supersede Tenant B's frozen candidate.

When the same application build serves several tenants, perform acceptance/freeze/verification for each tenant participating in production cutover.

36. Recommended Production Deployment Sequence

A typical controlled release sequence is:

```
php artisan migrate --force

php artisan db:seed \
  --class=DefaultChartOfAccountsSeeder \
  --force

php artisan db:seed \
  --class=PermissionSeeder \
  --force

php artisan permission:cache-reset
php artisan optimize:clear

npm ci
npm run build
npm run type-check

php artisan erp:backup:create --verify

php artisan erp:production-readiness --tenant=1
php artisan erp:deploy:preflight --tenant=1
php artisan erp:acceptance --tenant=1

php artisan erp:release:freeze 1.0.0-rc.1 --tenant=1
php artisan erp:release:verify --tenant=1
```

Do not continue when a blocking gate fails.

37. Maintenance-Mode Cutover

A typical maintenance sequence is:

```
php artisan erp:maintenance enable
```

```
php artisan queue:restart
php artisan migrate --force
```

```
npm ci
npm run build
```

```
php artisan optimize
php artisan queue:restart
```

```
php artisan erp:maintenance disable
php artisan erp:health
```

Adjust commands to the actual release mechanism.

Do not run:

```
composer update
```

during a production cutover.

Deploy the locked dependency set.

38. Post-Deployment Verification

After deployment:

```
php artisan erp:release:verify --tenant=1
php artisan erp:health
php artisan erp:deploy:preflight --tenant=1
php artisan erp:acceptance --tenant=1
```

Then manually verify representative workflows.

Purchasing

```
Purchase Order
-> Goods Receipt
-> Supplier Invoice
-> Supplier Payment
```

Sales

```
Sales Order
-> Allocation
-> Dispatch
-> Sales Invoice
-> Customer Receipt
```

Returns / AR

Sales Return / Credit Note
-> Credit Application / Refund

Treasury

Treasury Transfer
-> Bank Statement
-> Bank Reconciliation

Financial Reporting

Verify:

- Trial Balance;
- Profit & Loss;
- Balance Sheet;
- Cash Flow.

Also verify:

- login;
 - tenant context;
 - branch restrictions;
 - CSV export;
 - XLSX export;
 - scheduler heartbeat;
 - queue heartbeat;
 - logs;
 - next scheduled backup.
-

39. Production Acceptance Remediation Principles

When an acceptance check fails:

Tenant/branch mismatch

- inspect the reported records;
- identify the originating workflow;
- fix the workflow first;
- never mass-rewrite tenant IDs without an audited repair plan.

Journal integrity

- trace the source document;
- inspect the journal;
- do not manually edit posted lines;
- use reversal or compensating accounting workflows.

AR/AP open items

- inspect source invoices/payments/receipts;
- review allocation history;
- repair using settlement workflows or explicit audited repair commands.

Inventory balance

- trace product/warehouse stock ledger;
- fix originating movement logic;
- use audited reconciliation rather than direct posted-balance editing.

Purchase Return reservation

Inspect:

```
accepted_quantity
returned_quantity
return_reserved_quantity
```

and ensure:

```
returned_quantity
+ return_reserved_quantity
<= accepted_quantity
```

40. Security Rules

Production expectations include:

- APP_DEBUG=false;
 - HTTPS;
 - secure session cookies;
 - strong production secrets;
 - least-privilege database credentials;
 - protected tenant files;
 - backend authorization;
 - no trust in frontend permission visibility;
 - tenant-safe route-model binding;
 - no sensitive secrets in logs;
 - no cross-tenant cache keys;
 - no cross-tenant background jobs.
-

41. Audit and Immutability

Preserve the project's existing auditing behavior.

Financial and stock-impacting events should remain traceable.

Avoid direct edits to:

- posted journals;
- posted invoices;
- posted receipts/payments;
- stock ledger history;
- settlement history.

Use explicit business workflows for:

- reversal;

- cancellation;
 - adjustment;
 - reconciliation.
-

42. Migration Rules

Before creating a migration:

1. search existing migrations;
2. confirm another migration does not already own the same column;
3. confirm the current model schema;
4. confirm fresh migration ordering;
5. avoid duplicate column creation;
6. include tenant IDs where required;
7. include tenant IDs in relevant unique constraints.

Fresh migration safety matters as much as upgrading an existing database.

43. Route/Module Audit Rules

When doing a project-wide audit, prioritize:

1. missing/empty route files;
2. wrong namespace/path combinations;
3. files declaring the wrong class;
4. duplicate classes;
5. missing service dependencies;
6. undefined variables/imports;
7. tenant-unscooped critical queries;
8. wrong database column names;
9. fresh-migration conflicts;
10. duplicated scheduler registrations;
11. blocking deployment/acceptance failures.

Warning-only hardening issues can be reviewed separately and do not need to block normal development unless their risk becomes material.

44. Current Audit Status

The recent full ERP audit focused on critical/high functional and production-integrity defects.

Major areas addressed include:

- missing Sales Invoice route registration;
- service namespace/path mismatches;
- files declaring incorrect model classes;
- Treasury accounting gateway duplication/misplacement;
- missing AR accounting integration services;

- customer open-item state helpers;
- missing LogicException import in Customer Refund;
- undefined AR report filters;
- tenant-scoped Production Acceptance;
- correct GL journal-line column names;
- migration ownership conflicts;
- duplicate scheduled commands;
- inventory acceptance integrity;
- Purchase Return reservation integrity;
- GL header-to-line reconciliation;
- release-candidate tenant isolation;
- fail-closed production commands when no active tenant is validated.

At the current project decision point:

Confirmed critical/high functional findings are treated as complete; warning-only items may be deferred.

Before a real production launch, rerun all blocking production gates on the final codebase.

45. Important Development Rule for Future Sessions

Whenever development is continued from a new ChatGPT conversation:

1. upload the latest complete ERP project ZIP;
2. treat that ZIP as authoritative;
3. inspect it before writing new code;
4. do not repeat completed steps;
5. do not rely on old code snippets when the ZIP differs;
6. preserve the current architecture and naming;
7. fix verified integration defects before adding dependent features;
8. work one complete logical development step at a time.

The project itself is the source of truth.

46. Useful Command Reference

Development

```
composer install
npm ci
php artisan serve
npm run dev
```

Quality checks

```
php artisan optimize:clear
php artisan route:list
php artisan schedule:list
npm run type-check
npm run build
```

Permissions / foundation

```
php artisan db:seed --class=DefaultChartOfAccountsSeeder
php artisan db:seed --class=PermissionSeeder
php artisan permission:cache-reset
```

Queue

```
php artisan queue:work \
  --queue=notifications,exports,default \
  --tries=3 \
  --timeout=1200
```

```
php artisan queue:restart
```

Scheduler

```
php artisan schedule:work
```

Operations

```
php artisan erp:health
php artisan erp:production-readiness --tenant=1
php artisan erp:deploy:preflight --tenant=1
php artisan erp:acceptance --tenant=1
```

Backup

```
php artisan erp:backup:create --verify
php artisan erp:backup:verify BACKUP_ID
php artisan erp:backup:restore-check BACKUP_ID
```

Release candidate

```
php artisan erp:release:freeze 1.0.0-rc.1 --tenant=1
php artisan erp:release:verify --tenant=1
```

47. Final Engineering Principles

When changing this ERP, always protect these five invariants:

1. Tenant isolation

A tenant must never read, modify, validate, freeze, settle, post, or report another tenant's data.

2. Branch authorization

A user must never access a branch they are not authorized to access.

3. Accounting integrity

Every posted financial workflow must remain balanced, traceable, and reversible through controlled workflows.

4. Inventory integrity

Stock movements, balances, reservations, and returns must reconcile without silent quantity/value corruption.

5. Release integrity

Only a build that passes production gates and matches its frozen release fingerprint should be deployed.

Developer Handoff Checklist

Before handing this ERP to another developer or AI development session, record all of the following:

- authoritative project ZIP / commit;
- migration state and whether `migrate: fresh` is expected to work;
- seeders required for permissions, accounting defaults, SaaS plans/features, and Platform Admin;
- currently enabled tenant and branch used for testing;
- active document sequences for tested branches;
- queue and scheduler mode;
- current plan/subscription assigned to the test tenant;
- any temporary compatibility fixes not yet applied project-wide;
- failing acceptance scenario and exact stack trace if work is incomplete.

For code review, group changed files by category where possible:

```
Controllers
Requests
Models
Services
Policies
Routes / configuration
Vue pages / components
TypeScript types
Migrations / seeders
Tests
```

This makes changes easier to review and reduces the chance of missing a dependency.

48. Source-of-Truth Priority

When information conflicts, use this priority:

1. latest uploaded/current ERP source code;
2. current migrations and schema;
3. current project README and operational runbooks;
4. this developer guide;
5. older implementation notes or chat history.

Never modify current working code merely to make it match an outdated document.

End of ERP Developer Guide